

COMP202

Complexity of Algorithms

Probabilistic analysis and randomized algorithms

Reading materials: Appendix C.2, C.3, Chapter 7 in CLRS.

At the conclusion of this set of lecture notes, you should:

- 1 Understand basics of probability theory.
- 2 Be able to explain the mechanism behind the *randomised* Quick Sort algorithm, and *why* randomisation is used.
- 3 Be able to explain the mechanism behind the *randomised* Select algorithm and identify key differences with the randomised Quick Sort algorithm.

- Probability theory is useful in many contexts in the theory of algorithms.

It can be used to analyze the *average-case* performance of algorithms (by *weighting* the running time according to a probability distribution on inputs).

Randomized algorithms (those that use random bits during their execution to make decisions) are also becoming more commonplace and useful in recent years. Analysis of such randomized algorithms also requires use of probability theory.

See Appendix C in [CLRS] for details on probability theory!

Sample space

- *Sample space* is the set of all possible *outcomes* from an *experiment*.

Examples:

- 1 Rolling a fair (6-sided) die. The sample space is $\{1, 2, 3, 4, 5, 6\}$.
- 2 Flipping a fair coin. The sample space is $\{H, T\}$.
- 3 Flipping a fair coin until it comes up heads. Here the sample space is *infinite*, with the i^{th} possible *outcome* being a sequence of $i - 1$ tails followed by a single heads, for $i = 1, 2, 3, 4, \dots$

Probability spaces, Events

A *Probability space* is a sample space $\mathcal{S}$ and a *probability function* Pr , which maps subsets of $\mathcal{S}$ to $[0, 1]$.

Each subset, $\mathcal{A} \subseteq \mathcal{S}$, is called an *event*.

Probability function

A probability function, $\Pr$, possesses the following properties:

- ① $\Pr(\emptyset) = 0$,
- ② $\Pr(\mathcal{S}) = 1$,
- ③ $0 \leq \Pr(\mathcal{A}) \leq 1$, for any $\mathcal{A} \subseteq \mathcal{S}$,
- ④ If $\mathcal{A}, \mathcal{B} \subseteq \mathcal{S}$ and $\mathcal{A} \cap \mathcal{B} = \emptyset$, then

$$\Pr(\mathcal{A} \cup \mathcal{B}) = \Pr(\mathcal{A}) + \Pr(\mathcal{B}).$$

An example of a probability space

Consider the sample space earlier, namely that of flipping a fair coin until it comes up heads.

We can represent the sample space as an infinite set of strings, namely $\{H, TH, TTH, TTTH, \dots\}$.

The probability function on this sample space assigns these probabilities to events:

$$Pr(H) = 1/2$$

$$Pr(TH) = 1/4$$

$$Pr(TTH) = 1/8$$

...

$$Pr(H \text{ or } TTH) = 1/2 + 1/8 = 5/8$$

...

(These statements make use of the *independence* of coin flips discussed on the next slide, and the last statement uses the final property of the probability function on the previous slide.)

Independent Events

- Two events are *independent* if $Pr(\mathcal{A} \cap \mathcal{B}) = Pr(\mathcal{A}) \cdot Pr(\mathcal{B})$.
- A collection of events $\{\mathcal{A}_1, \mathcal{A}_2, \dots, \mathcal{A}_n\}$ is *mutually independent* if

$$Pr(\mathcal{A}_{i1} \cap \mathcal{A}_{i2} \cap \dots \cap \mathcal{A}_{ik}) = Pr(\mathcal{A}_{i1}) \cdot Pr(\mathcal{A}_{i2}) \cdot \dots \cdot Pr(\mathcal{A}_{ik})$$

for any subcollection of events $\{\mathcal{A}_{i1}, \dots, \mathcal{A}_{ik}\}$.

Example

Let $\mathcal{A}$ be the event that the roll of a fair die is 6, and let $\mathcal{B}$ be the event that the roll of a second fair die is a 3, and let $\mathcal{C}$ be the event that the sum of the two dice is 10.

Then $\mathcal{A}$ and $\mathcal{B}$ are independent events, but it's easy to check that $\mathcal{C}$ is not independent with either $\mathcal{A}$ or $\mathcal{B}$.

This follows since

$$Pr(\mathcal{A}) = Pr(\mathcal{B}) = 1/6$$

$$Pr(\mathcal{C}) = 3/36$$

$$Pr(\mathcal{A} \cap \mathcal{C}) = 1/36 = Pr(\mathcal{A} \cap \mathcal{B}), \text{ and}$$

$$Pr(\mathcal{B} \cap \mathcal{C}) = 0.$$

Conditional Probability

The *conditional probability* of an event $\mathcal{A}$ occurring, given another event $\mathcal{B}$, denoted $Pr(\mathcal{A}|\mathcal{B})$ is

$Pr(\mathcal{A}|\mathcal{B}) = Pr(\mathcal{A} \cap \mathcal{B})/Pr(\mathcal{B})$, assuming that $Pr(\mathcal{B}) > 0$.

In other words, conditional probability talks about the probability of some event occurring, *given the knowledge that some other event has already occurred*.

Examples

- ① $\mathcal{A}$ is event that two throws of a fair die sum to 10, $\mathcal{B}$ is event that the first throw is 6.

$$Pr(\mathcal{B}) = 1/6 \text{ and } Pr(\mathcal{A} \cap \mathcal{B}) = 1/36.$$

$$\text{Thus, } Pr(\mathcal{A}|\mathcal{B}) = (1/36)/(1/6) = 1/6.$$

- ② Suppose that you have a red die and a black die.
Let $\mathcal{C}$ be the event that after throwing the two dice they sum to 9, and $\mathcal{D}$ be the event that the red die shows a 5 or 6.

Then (as you can verify, think about it!!!):

$$Pr(\mathcal{D}) = 2/6$$

$$Pr(\mathcal{C} \cap \mathcal{D}) = 2/36$$

Hence,

$$Pr(\mathcal{C}|\mathcal{D}) = Pr(\mathcal{C} \cap \mathcal{D})/Pr(\mathcal{D}) = (2/36)/(2/6) = 1/6.$$

Also, $Pr(\mathcal{C}) = 4/36$. Hence,

$$Pr(\mathcal{D}|\mathcal{C}) = Pr(\mathcal{D} \cap \mathcal{C})/Pr(\mathcal{C}) = (2/36)/(4/36) = 1/2.$$

- A *random variable* is a function X that maps outcomes from some sample space to real numbers.

In algorithm analysis we often use a random variable X with a discrete set of outcomes to characterize the running time of a randomized algorithm.

- The *expected value* of a discrete random variable X is defined as $\mathbb{E}(X) = \sum_i i \cdot \Pr(X = i)$ where the summation is over the range of possible values of X .

The expected value is *linear*, meaning that for any two random variables X and Y we have $\mathbb{E}(X + Y) = \mathbb{E}(X) + \mathbb{E}(Y)$.

Example

Consider the event of rolling a fair die, and let X denote the value that occurs. Then we have

$$Pr(X = 1) = Pr(X = 2) = \dots = Pr(X = 6) = 1/6.$$

By definition, we have

$$\mathbb{E}(X) = 1 \cdot \frac{1}{6} + 2 \cdot \frac{1}{6} + \dots + 6 \cdot \frac{1}{6} = 3\frac{1}{2}.$$

Example (cont.)

Suppose that Y is the value obtained by the roll of another fair die. Then $X + Y$ represents the sum of the two dice.

We have

$$\mathbb{E}(X + Y) = \mathbb{E}(X) + \mathbb{E}(Y) = 3\frac{1}{2} + 3\frac{1}{2} = 7,$$

which can also be computed from the definition of expected value using the facts that

$$Pr(X + Y = 2) = 1/36$$

$$Pr(X + Y = 3) = 2/36$$

...

$$Pr(X + Y = 6) = 5/36$$

$$Pr(X + Y = 7) = 6/36$$

$$Pr(X + Y = 8) = 5/36$$

...

Another example

Consider the problem from before of flipping a fair coin until it comes up heads for the first time.

Let X denote the *number* of flips that must be made until the heads appears (including the final flip). Then X is a random variable that takes values in $\{1, 2, 3, \dots\}$.

We note that

$$\Pr(X = i) = \left(\frac{1}{2}\right)^i \text{ for } i \in \{1, 2, 3, \dots\}.$$

By definition, we then have

$$\mathbb{E}(X) = \sum_{i=1}^{\infty} i \cdot \Pr(X = i) = \sum_{i=1}^{\infty} i \cdot \left(\frac{1}{2}\right)^i.$$

Using a bit of calculus (which I don't necessarily expect you to know), we can find that $\mathbb{E}(X) = 2$.

Another way to get the same result

We actually don't need to use calculus for this. If you consider the first coin toss, either it comes up heads (in which case the experiment is over), or it doesn't (which “costs” us one coin toss, plus however many more we need until the first heads shows up).

Thus we can write

$$\begin{aligned}\mathbb{E}(X) = & \Pr(X \text{ is heads}) \cdot 1 + \\ & \Pr(X \text{ is tails}) \cdot (1 + \text{expected number of additional coin} \\ & \text{tosses until first heads appears}).\end{aligned}$$

But the “expected number of additional coin tosses until the first heads appears” is exactly the same as $\mathbb{E}(X)$. (Having first obtained a “tails” the experiment “resets itself” and we're just interested in obtaining the first heads again.)

Thus we have

$$\mathbb{E}(X) = \Pr(X \text{ is heads}) \cdot 1 + \Pr(X \text{ is tails}) \cdot (1 + \mathbb{E}(X)).$$

Of course,

$$\Pr(X \text{ is heads}) = \Pr(X \text{ is tails}) = \frac{1}{2}, \text{ so}$$

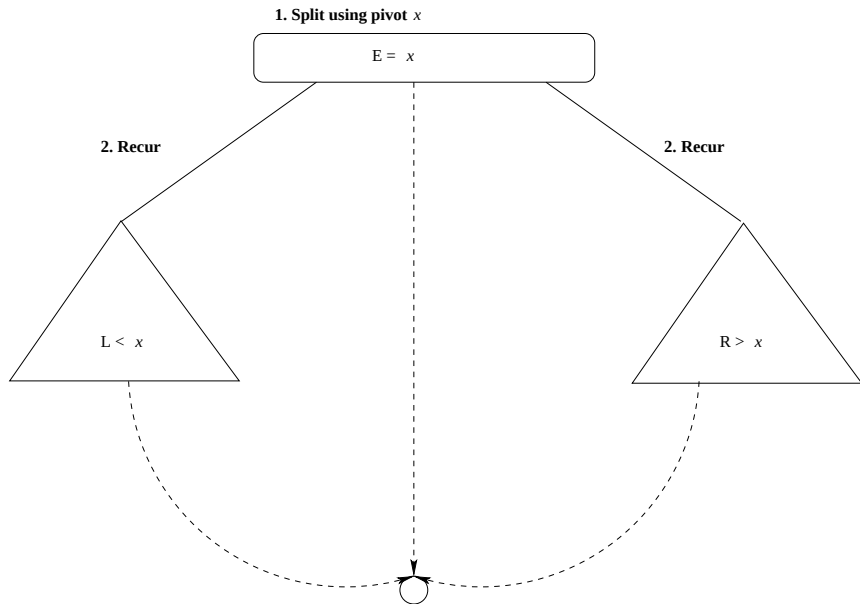
$$\mathbb{E}(X) = \frac{1}{2} \cdot 1 + \frac{1}{2} \cdot (1 + \mathbb{E}(X))$$

which upon solving for $\mathbb{E}(X)$ gives us the same result, namely $\mathbb{E}(X) = 2$.

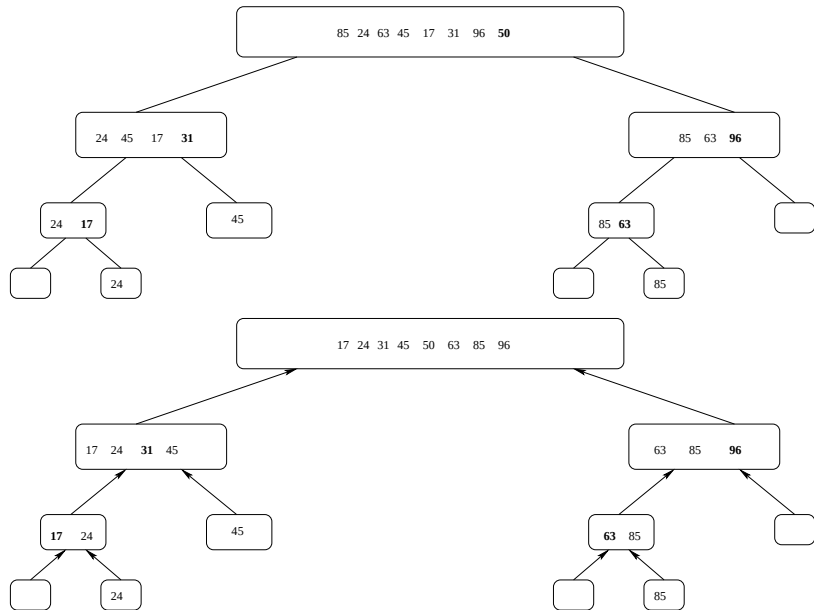
The QuickSort algorithm is a divide-and-conquer method for performing sorting. This method differs from MergeSort in that the steps for QuickSort are as follows:

- *Divide*: If $|S| > 1$, select a *pivot element* x in S and create three sequences: L , E , and G where
 - L stores elements in S that are less than x ,
 - E stores elements in S that are equal to x , and
 - G stores elements in S that are greater than x .
- *Recur*: Recursively sort the sequences L and G .
- *Conquer*: Put sorted elements from L , E , and G together to form a sorted list.

QuickSort Tree



QuickSort - Example



What is the running time of the QuickSort algorithm?

- Let s_i be the sizes of the input nodes (at depth i) in the tree.
- $s_i \leq n - i$. (Note that $s_i = n - i$, when use of *pivots* leads to a single empty sequence (for L or G).)
- The worst case complexity is bounded by

$$O(\sum_{i=0}^{n-1} s_i) = O(\sum_{i=0}^{n-1} n - i) = O(\sum_{i=0}^{n-1} i).$$

Since

$$\sum_{i=0}^{n-1} i = \frac{(n-1)n}{2}$$

the *worst case* running time of QuickSort is $O(n^2)$.

The worst case running time of QuickSort occurs when one of the sublists is in fact empty.

To avoid this worst case run time, we want to ensure that each of L and G are sufficiently large for each recursive call.

For any *deterministic* method of choosing a pivot element, we can design some list so that the running time is (almost) n^2 .

- Solution: Select a pivot element randomly!

Randomized QuickSort (cont.)

Recall that our proposed method for the randomized QuickSort algorithm is to choose the pivot element at random.

We can repeat the selection until we find a “good” pivot element (defined below). With a good pivot element during each step, we have an algorithm with (expected) running time that is $o(n^2)$.

Theorem The expected running time of randomized QuickSort (pivot chosen at random) is $O(n \log n)$.

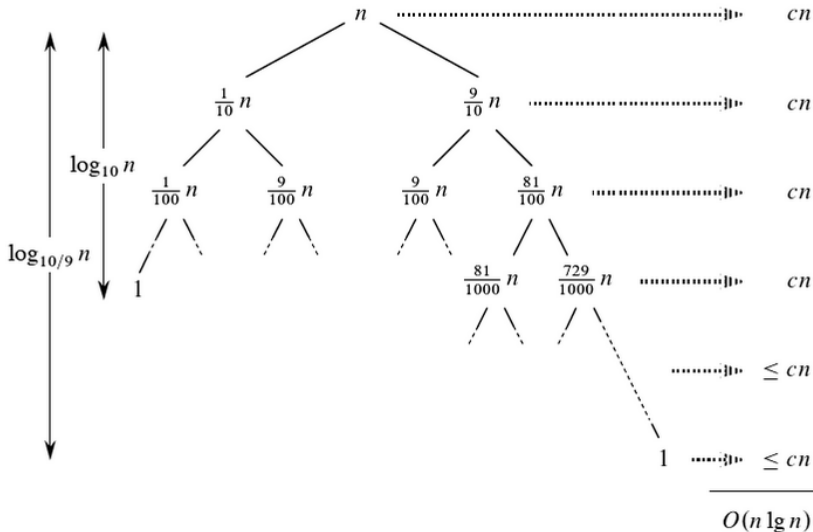
Proof idea:

- We say that a random pivot is *good* if the size of neither L or G is larger than $\frac{3}{4}|S|$.

Randomized QuickSort (cont.)

- The probability of success of choosing a good pivot is $\frac{1}{2}$.
- Fact: The *expected* number of times that a *fair* coin must be tossed until it shows heads k times is $2k$. Choosing a good pivot is analogous to flipping a fair coin until it comes up heads. So, on average we need two attempts (random choices) to find a good pivot.
- Any path in the QuickSort algorithm can contain at most $\log_{4/3} n$ levels with good pivots.
- Hence, the expected length of each path will be $2 \log_{4/3} n$.

Randomized QuickSort



- **Selection problem:** Given an (unsorted) list/array A of n numbers and an integer $i \in \{1, \dots, n\}$, find the i 'th *smallest* element.
- Special cases:
 - $i = 1$: minimum, $i = n$: maximum, $i = \lceil n/2 \rceil$: median.
- Naïve method: sort A and return the i th element. Running time: $O(n \log n)$.
- Question: can we do better if we only want *one* order statistic?
- Answer: **yes**, *Randomized-Select* runs in **expected** $O(n)$ time.

A special case

- Suppose $i = n - 1$, that is, you want the second largest element.
- You can find the maximum in worst-case $n - 1$ comparisons as we saw in Week 1.
- Then among all the other $n - 1$ elements you can find the maximum (which is the overall second largest) in worst-case $n - 2$ comparisons.
- Total of $2n - 3$ comparisons, and each comparison takes $O(1)$ time, so we get an $O(n)$ algorithm, which can't be improved upon.
- But if you care about the leading constant, there's actually a much faster algorithm.

Randomized Select

- Randomized-Select is closely related to Randomized QuickSort.
- Pick a **uniformly random pivot** $x \in A$ and partition:

$$L = \{a \in A : a < x\}, E = \{a \in A : a = x\}, G = \{a \in A : a > x\}.$$

- Let $k = |L| + 1$ be the rank of the pivot (assuming distinct keys for simplicity).
- If $i = k$, return x .
- If $i < k$, recurse on L ; if $i > k$, recurse on G with index $i - k$.
- Key difference with QuickSort: we recurse on *only one* side, whereas in QuickSort we recursed on both sides.

Randomized Select, pseudocode

RANDOMIZED-SELECT(A, i)

if $|A| = 1$ **then return** the only element of A

choose pivot x uniformly at random from A

partition A into L, E, G where $L < x$, $E = x$, $G > x$

$k \leftarrow |L| + 1$ // rank of pivot

if $i = k$ **then return** x

else if $i < k$ **then return** **RANDOMIZED-SELECT**(L, i)

else return **RANDOMIZED-SELECT**($G, i - k$)

Running time analysis

- Worst case: the pivot is always the smallest/largest element.
 - Subproblem sizes: $n - 1, n - 2, \dots, 1$
 - Total time: $O(n^2)$
- But the pivot is chosen **uniformly at random**.
- Intuition: with constant probability the pivot is “good” (not too close to the ends), so the recursive subproblem shrinks by a constant factor.

Theorem: The expected running time of Randomized Select is $O(n)$.

Proof idea:

- Just as in the analysis of QuickSort, we say that a random pivot is *good* if the size of neither L or G is larger than $\frac{3}{4}|S|$.

Deterministic Select

- There is also a **deterministic** selection algorithm with **worst-case** $O(n)$ running time.
- Main idea:
 - 1 Split A into groups of 5 elements.
 - 2 Compute the median of each group.
 - 3 Recursively find the median of these medians; call it m .
 - 4 Use m as pivot and partition A into L, E, G .
- One can prove m is always a “good pivot”.
- In practice: Randomized Select is simpler and usually faster; deterministic select is mainly of theoretical interest.